TEMA 03 · PROJETO INTEGRADOR DE APIS

PEDIDOJÁ

Acompanhamento de pedidos com notificações automáticas pelo Telegram

Node.js / PostgreSQL / Telegram Bot API / ViaCEP / AWS

O cliente quer acompanhar sem precisar perguntar

Mais transparência no e-commerce, menos mensagens manuais na loja.

Problema

Consultas repetidas sobre pagamento, separação, envio e entrega.

Solução

API registra mudanças e dispara notificações pelo Telegram.

Benefício

Histórico auditável e acompanhamento rápido do pedido.

Cinco etapas. Uma mensagem a cada avanço.

As transições são sequenciais e registradas no PostgreSQL.

1 **RECEBIDO**
Pedido criado

2 **PAGO**
Pagamento registrado

3 **SEPARADO**
Preparação concluída

4 **ENVIADO**
Saiu para entrega

5 **ENTREGUE**
Pedido finalizado

→ Cada transição cria histórico e enfileira a notificação do cliente.

APIs, bot e banco trabalhando juntos

Separação de responsabilidades: web, API, banco e worker.

PAINEL WEB

Cadastro e operação

REST API

Regras e validações

POSTGRESQL

Dados + migrations

WORKER

Fila + Telegram

ViaCEP valida endereços • Telegram envia/recebe mensagens • Webhook HTTPS registra eventos

Cadastrar clientes e produtos

ViaCEP completa o CEP e o cliente opta por receber via bot.

1. Cliente

Nome, e-mail, CEP, número e endereço validado pelo ViaCEP.

2. Produto

Nome, descrição e preço armazenado em centavos.

3. Telegram

Cliente abre o link exclusivo e toca em INICIAR.

Criar pedido, avançar status e confirmar

A cada etapa, o cliente pode confirmar ciência pelo Telegram.

Criar pedido

Selecionar cliente, item e quantidade. Total calculado no banco.

Avançar etapa

Pago → Separado → Enviado → Entregue. Sem pular etapas.

Confirmar ciência

Clique no botão volta pelo webhook e registra “respondida”.

Webhook protegido. Dados rastreáveis.

Os resultados ficam documentados, inclusive as falhas.

Autenticação

Segredo no cabeçalho do Telegram; sem ele, HTTP 401.

Idempotência

update_id único evita processar o mesmo evento duas vezes.

Testes

10 testes unitários aprovados; CI executa testes e lint.

Estados da mensagem: ENFILEIRADA · ENVIADA · FALHA · RESPONDIDA

Arquitetura planejada na AWS

Evidências reais devem ser coletadas após configurar contas, DNS e domínio.

Amazon EC2

API e worker em containers; Caddy na frente com HTTPS.

Amazon RDS

PostgreSQL gerenciado, privado, acesso só via aplicação.

Route 53 + Caddy

DNS em domínio próprio e certificado Let’s Encrypt.

Custo: dimensionar recursos e configurar alertas de US$ 25, US$ 50 e US$ 100.

PedidoJá: da criação do pedido até a entrega

Integração real de APIs, persistência, webhooks e notificações.

API própria + API pública + Telegram + PostgreSQL + Docker + plano de publicação HTTPS

Obrigado! • Perguntas?